University of Tlemcen Abou Bekr Belkaid
Sciences faculty
Computer Science Department
2025/2026

ING 3 IA --- Mobile Applications Practicum

Practical 3: Validation & Navigation between screens (Routing)

1) Introduction :

The aims of this practical are as follows:

- ✓ To define classes representing data models.
- ✓ To manipulate *Navigator* class methods to navigate between the different screens of the ***"meal_planner"*** app.
- ✓ To learn how to validate the value entered in a *TextField* widget.

2) Data models creation: in the "*lib*" folder, add a new sub-folder named "*models*". This latter will contain all data models used in the app.

a) The "Meal" data model: in the "*models*" folder, add a new file named "*meal.dart*". In this file, define a class named "*Meal*" with three required attributes: "*name*" and "*imgPath*" of type *String*, and "*listOfIngredient*" of type *List<String>*. Additionally include an optional attribute named "*identifier*" of type *String*. Finally, provide the corresponding constructor.

b) The "MealsOfADay" data model: in the "*models*" folder, add a new file "*meals_of_a_day_meals.dart"*. In this file, define a class "*MealsOfADay*" with two required attributes: "*day*" of type *String* and "*listOfMealsForADay*" of type *List<Meal>*. Finally, define the corresponding constructor.

c) **The input model:** add some images to your project and specify their path in the "*pubspec.yaml*" file. Then, in the "*HomeScreen*" page, modify the Dart list "*week_days_List*" so that it contains 7 elements of type "*MealsOfADay*" where each element represents one day of the week with its list of meals (List<Meal>). For the widget *WeekDaysCard,* replace the *day* parameter value by "*week_days_List[index].day*".

3) **Navigation (ref. course):** in this section, we will implement navigation between the different screens of the ***"meal_planner"*** app. The navigation schema is shown in *Figure 1*.

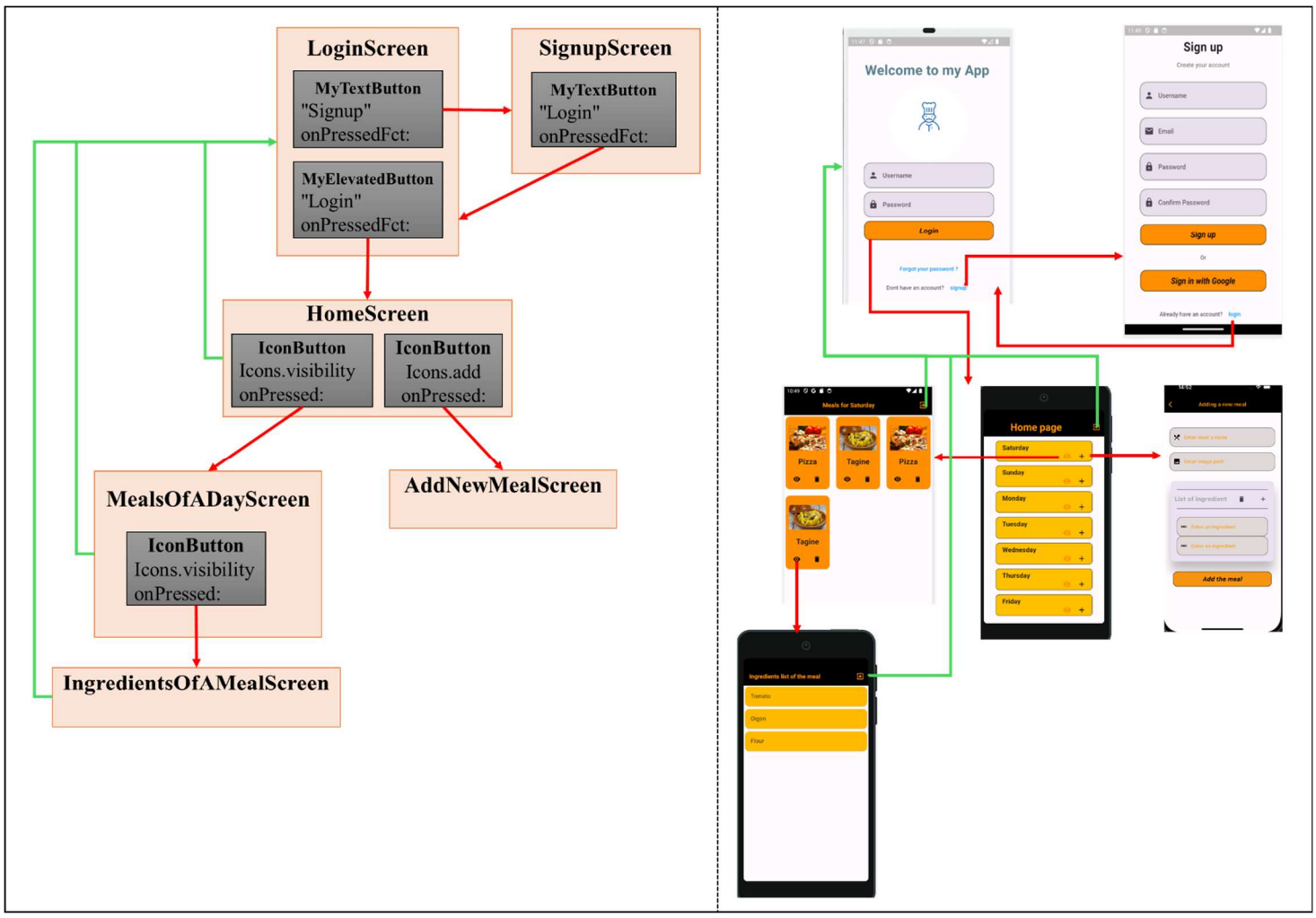

Figure 1 : ***Navigation schema***

a) **Simple Navigation:** using **static** navigation.

1. In the file "*main.dart*", add the parameter "*routes*" to *MaterialApp* then define the route for each page ("*HomeScreen*", "*LoginScreen*", "*SignupScreen*", "*MealsOfADay*", "*AddNewMealScreen*" and "*IngredientsOfAMealScreen*").
2. Specify the "*home*" property of *MaterialApp* so that "*LoginScreen*" is the first page displayed when the app starts.
3. From the *onPressedFct* parameter of the "*signup*" text button of "*LoginScreen*" page, navigate to "*SignupScreen*" page using ***pushReplacementNamed*** method of *Navigator* class. The same goes for navigation from "*SignupScreen*" page to "*LoginScreen*" page through "*login*" text button.
4. Also, use the ***pushReplacementNamed*** method to navigate to the "*HomeScreen*" page from the *onPressedFct* parameter of the "*Login*" elevated button in the "*LoginScreen*" page.
5. On the "*HomeScreen*","*IngredientsOfAMealScreen*", and "*MealOfADayScreen*" pages, implement the *AppBar action* button (***Icons.exit_to_app***) to navigate to the "*LoginScreen*" using the ***pushNamedAndRemoveUntill*** method with the condition *(route) => false* to delete all existing routes in the stack.

b) **Navigation with data passing :**

1. Modify "*WeekDaysCard*" class to have a single attribute named "*dayAndItsMealsList*" of type "*MealsOfADay*". The latter will replace the attribute "*day*" of type *String*.
2. From the "*WeekDaysCard*", navigate to the "*MealsOfADayScreen*" through the *onPressed* parameter of the "*visibility*" icon button, and to the "*AddNewMealScreen*" through the *onPressed* parameter of the "*add*" icon button. In both cases use the ***pushNamed*** method.

3. To display the list of meals for a specific day in the grid of the "*MealsOfADayScreen*" page, the *"dayAndItsListOfMeals*" attribute must be passed as an argument to the ***pushNamed*** method when navigating to "*MealsOfADayScreen*".This attribute is recovered from the "*MealsOfADayScreen*" page as follows:

```
final dayAndItsListOfMeals =
   ModalRoute.of(context) ?.settings.arguments as MealsOfADay ;
```

   This will enable the "*MealsOfADayScreen*" to retrieve the value of the "***meal****"* attribute of the "*MealCard*".

4. In the "*MealCard*", use the ***pushNamed*** method in the *onPressed* parameter of the "*visibility*" icon button to navigate to the "*IngredientsOfAMealScreen*". Pass the "***meal****"* attribute as an argument, which will be retrieved in a variable named *currentMeal*.
5. Pressing the "*Add the meal*" elevated button of the "*AddNewMealScreen*" page should create a new object of type *"Meal"*. The object's *name*, *image path*, and *list of ingredients* are retrieved from the *TextEditingController* (*TFController*) associated with the custom *MyTextField* widget. Noting that the list of ingredients can be retrieved from the local variable *"listOfTextField"*. Finally, the newly created object is passed back to the *"WeekDaysCard"* widget using the ***pop*** method of the *Navigator* class.

4) Validation of input data :

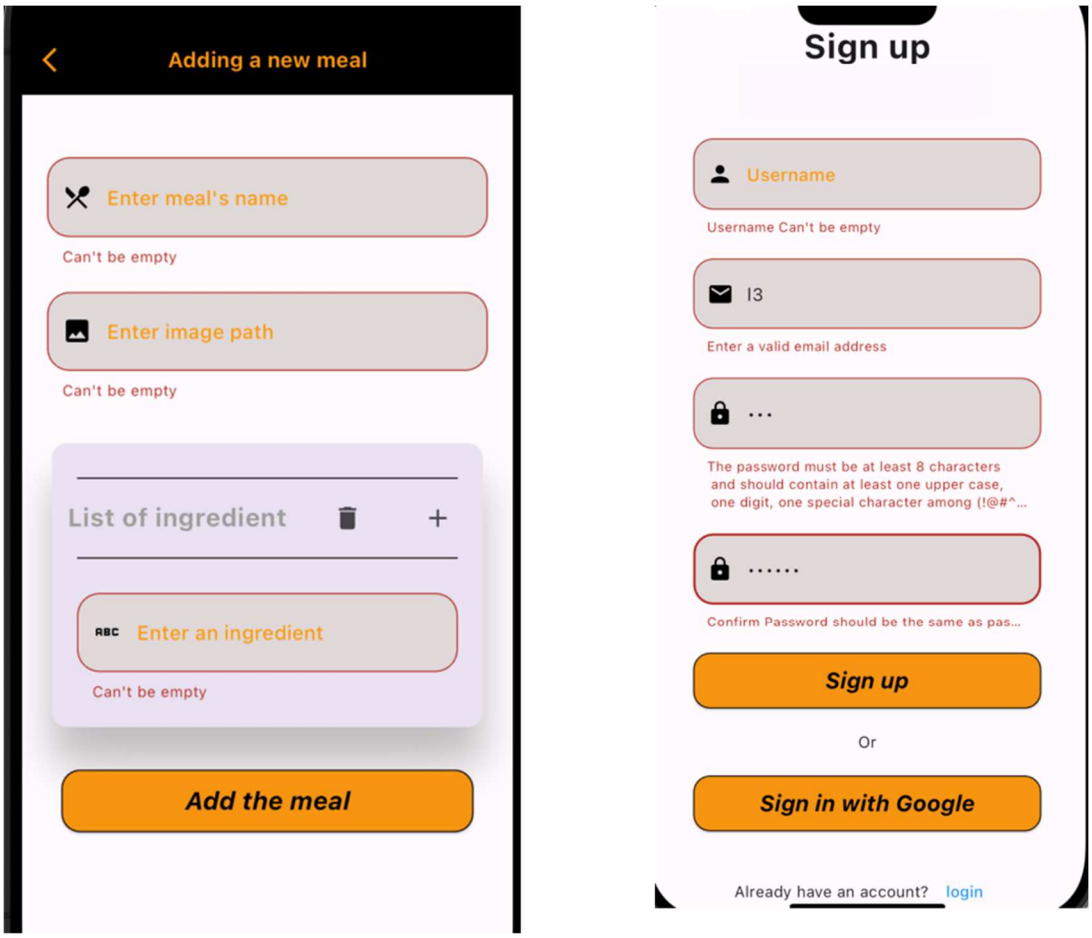

Figure 2 : Input data v*alidation*

To ensure that all the text field (i.e. *MyTextField*) within the "*LoginScreen*", "*SignupScreen*" and "*AddNewMealScreen*" pages are properly filled (see Figure 2), it is necessary to:

i. Wrap "*Column*" widget inside a "*Form*" widget, which requires a "*key*" parameter of type "*GlobalKey<FormState>*". For example, *"Key: keyFormState"* where,

```
GlobalKey <FormState> keyFormState = GlobalKey <FormState>();
```

ii. Validate the input format of the "*Email*", "*Password*" and "*Confirm password*" text fields of the "*SignupScreen*" page. To achieve this, define the *"TFValidator"*

parameter of each *"MyTextField"* widget. For example, the *"TFValidator"* parameter for the "*Email*" text field can be implemented as follows:

```
TFValidator: (val) => emailValidationFct (val),
```

Where, ***"emailValidationFct"*** is a function defined as follow in the file *"/lib/helpers/validators.dart"*.

```
String? emailValidationFct (String? value) {
    const pattern =  r"^[a-zA-Z0-9.a-zA-Z0-9.!#$%&'*+-
      /=?^_`{|}~]+@[a-zA-Z0-9]+\.[a-zA-Z]+";
   final regex = RegExp(pattern);
  if (value!.isEmpty) { return "Email Can't be empty ";}
  else if (!regex.hasMatch(value)) { return 'Enter a valid email address';}
  return null;   }
```

For the "*Password*" text field, it is necessary to check if the password contains at least one uppercase letter, one digit and one special character among (@#\&*~). The function "***pwdValidationFct"*** is as follows:

```
String? pwdValidationFct (String? value) {
  const pattern =r'^(?=.*[A-Z])(?=.*?[0-9])(?=.*?[ @#\&*~]).{8,}';
  final regex = RegExp(pattern);
  if (value!.isEmpty) { return "Password Can't be empty "; }
else if (!regex.hasMatch(value)) { return 'The password must be at least 8
      characters \n and should contain at least one uppercase, \n one digit,
      one special character among (@#\&*~)'; }
  return null; }
```

Following the same logic, implement "***pwdConfirmValidationFct*** " function for "*Confirm*

password" text field. This function aims to check if the value of "*pwdConfirmController*" is the same as "*pwdController*" value.

```
TFValidator:(value)=> pwdConfirmValidationFct (value, pwdController.text),
```

The same applies to the text fields in the "*AddNewMealScreen*" and "*LoginScreen*", which must not be left blank.

iii. Verify the validity of all input values when clicking the "*Login*", "*Signup*" and "*Add new meal*" elevated buttons. To achieve this, the following check should be implemented within the *onPressedFct*: () { ... } parameter of each elevated button.

```
if(keyFormState.currentState !.validate()) {
    // Navigate to other screens…. } else {print("Not Valide");}
```

Note: The ***"fluttertoast"*** package can be used to display a non-blocking temporary message on the screen, alerting the user when the input data is invalid (see Figure 3)."

```
displayAToast() {
        Fluttertoast.showToast(
        msg: "Your entries are not valide",
        toastLength: Toast.LENGTH_SHORT,
        gravity: ToastGravity.CENTER,
        timeInSecForIosWeb: 1,
        backgroundColor: Colors.red,
        textColor: Colors.white,
        fontSize: 16.0,
    );
}
```

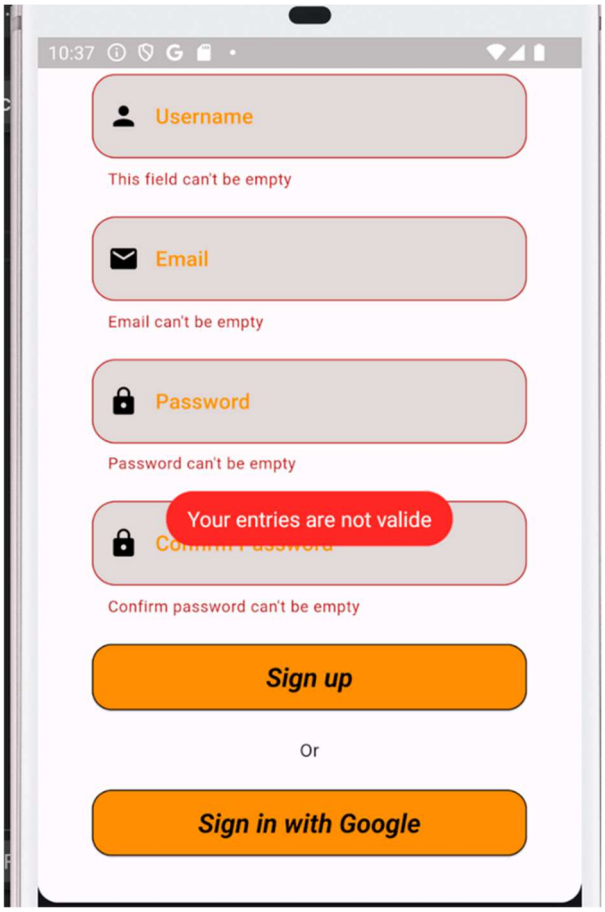

Figure 3 : Input data v*alidation*